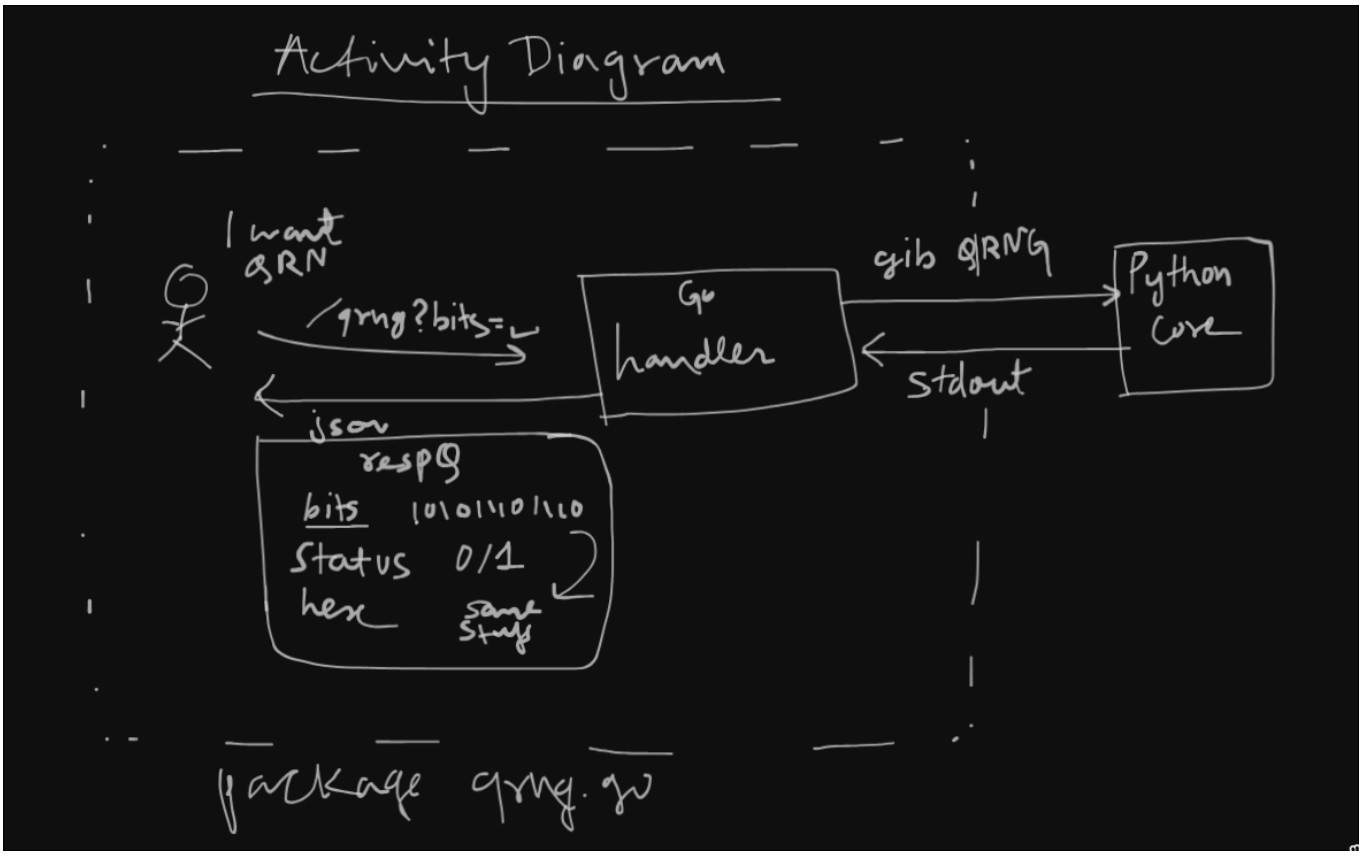
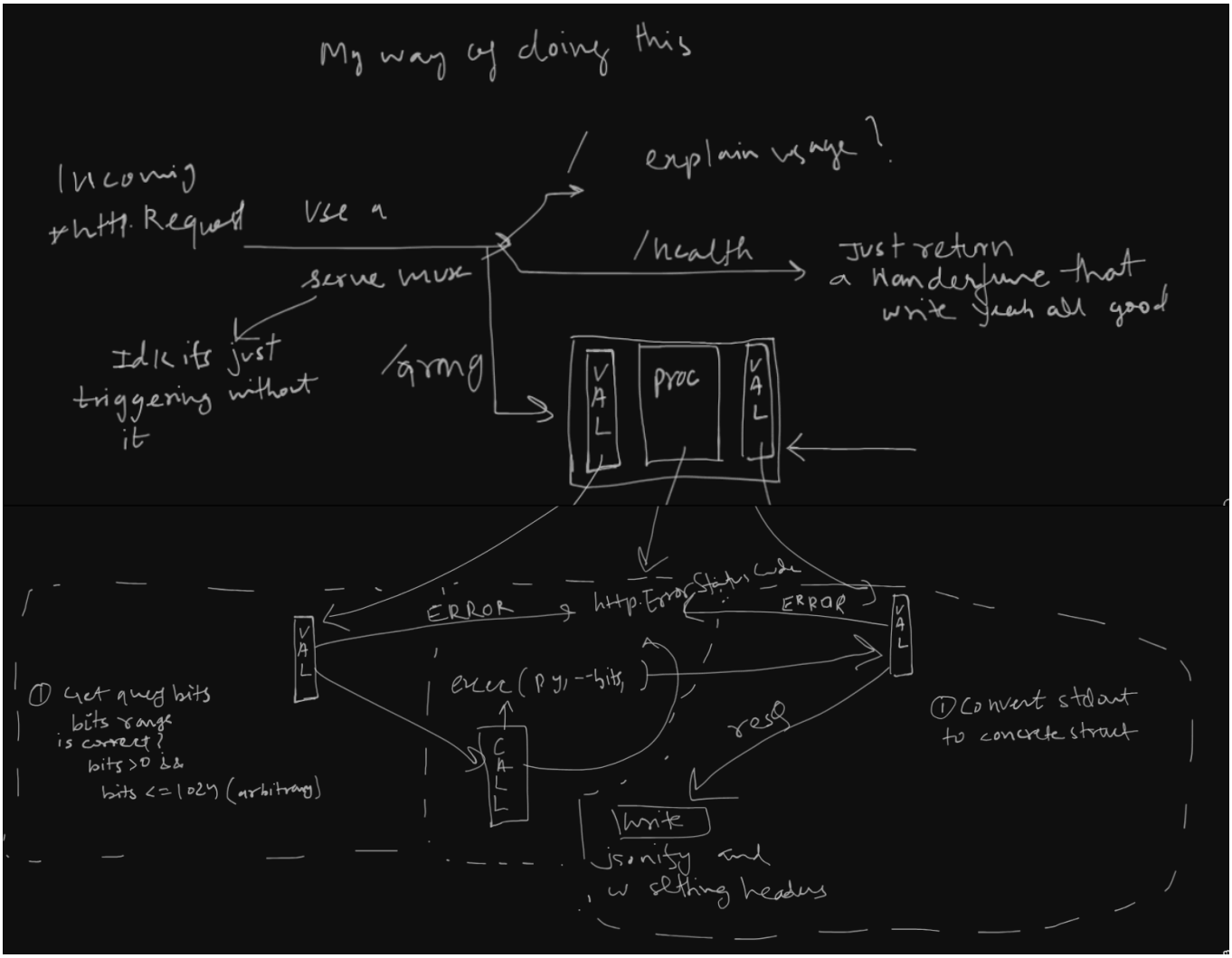
Entire Thingy



Go Handlers



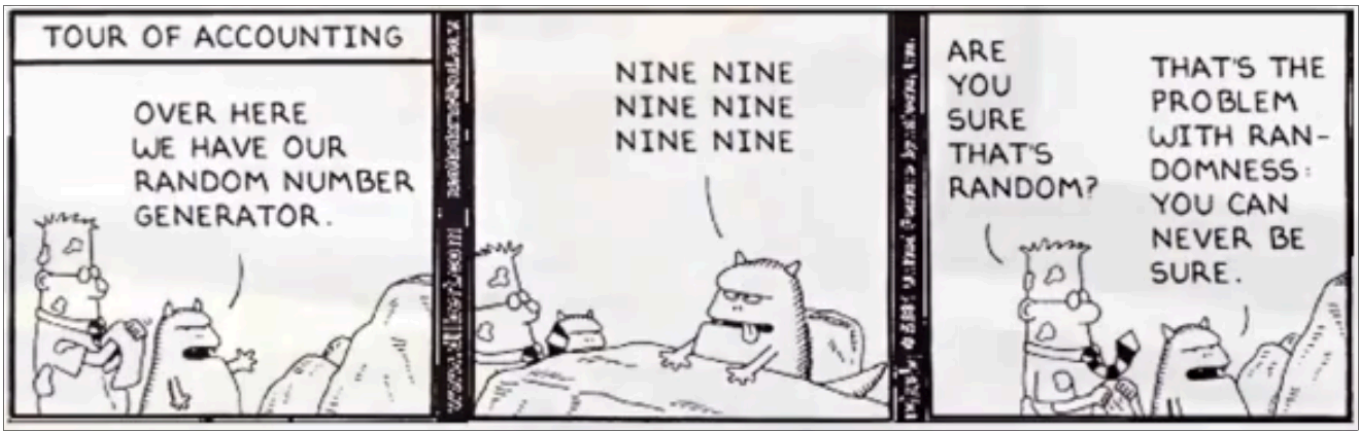
Python

Well this part is mostly mathematics, so I will try to explain the mathematics more simply here compared to the python notebook.
The big problem it aims to solve is that we need randomness. Its difficult for me to answer "How much randomness is enough randomness?", so I won't be answering that; instead I would just tell you one of the answers of why we need randomness. Almost all of cryptography is based on keys, keys are assumed to be random but the generators could be biased. If a generator gives bad keys (theres pattern in the keys), we have a situation which the encryption algorithm creators didn't consider. So the strength of encryption comes into question. I am yet to find a really satisfactory answer to this question.

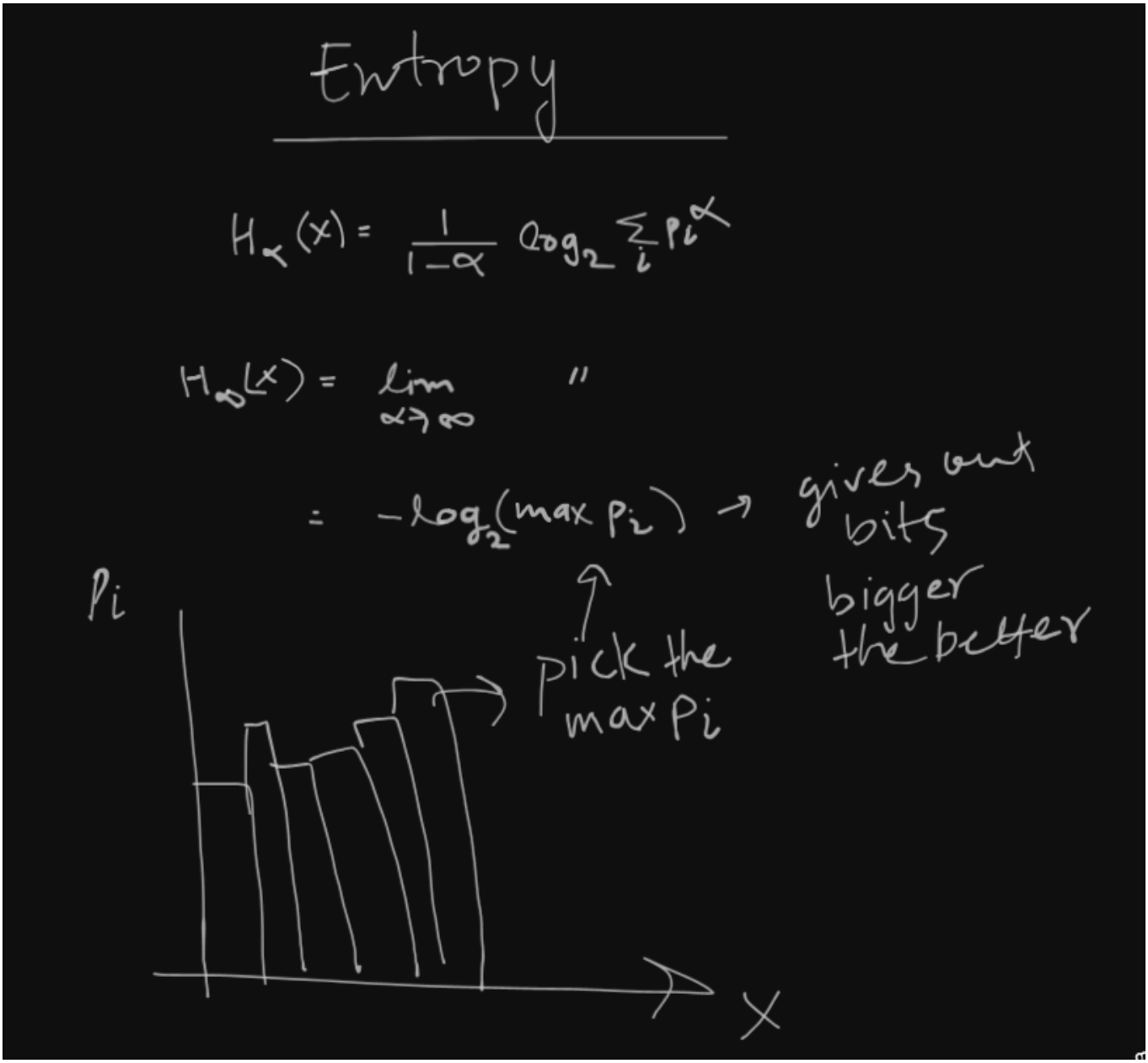
NOTE: This is just for demonstration purposes, I am actually just using AER simulator which tries to simulate the results of a quantum computer so it is infact PRNGish.

We are talking about generators here, the ones that churn out random numbers. They take in randomness from various physical stuff (deep dive) and use that to give out numbers, so its just a function we can think of it like that.
No matter what you do a function is deterministic, it has defined inputs and outputs. So our best

bet becomes to make our input random as possible or try to source from multiple sources, then take that output sort of go through a distillation process to get purified RNG. Thats really what we are doing.



Apart from some objective tests, I like to visualise random numbers in a spectrum and not rigid, i.e, theres no discrete value that says X is a PRNG. It could be pretty close to a TRNG or maybe even worse of a PRNG.
Entropy is a way to quantify randomness??



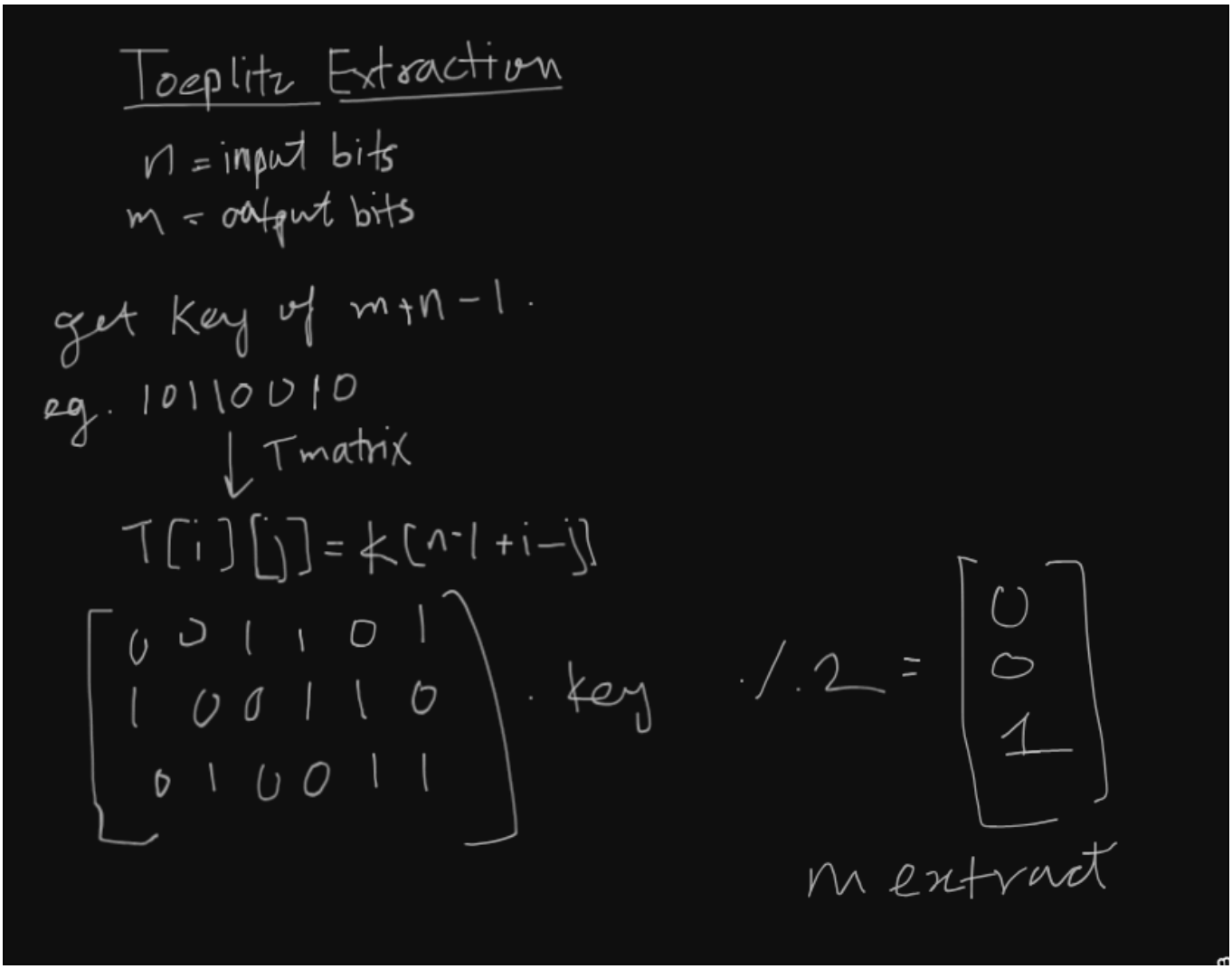
Min entropy is used here cuz its sort of like the big O notation of entropies. So it gives you the worst case (hence the name minimum entropy)

Note: I wrote gives out bits, bigger the better. The rule of thumb that I follow here is that if lets say result way 8 bits, that means its equally hard to predict the outcome as hard it to predict outcome of flipping 8 coins.

Here we are flipping a coin (hadmard gate), now to some noise or any misc reason if its more likely that 1 will come out lets say p_1 = 0.55. Then my min entropy decreases from 1. Thats bad. If we are doing it for 128 bits it'd be less that 128.

So to extract more randomness we can try and reduce the outcomes inside X, make it so that the p_max isn't 0.55 anymore.

Sum of all outcomes is fixed, so our probability distribution being uniform is our best bet.



There big maths going on behind this that proves that the extracted m is more uniformly random, but a rule of thumb is that XOR reduces biases. Lets say i have two coins, they are both biased 60% towards 1. So all my bits are slightly biased towards 1, that means in my probability distribution the outcomes which has more 1 will have slightly higher probability (exactly what happened in the ipynb graphs). The way xor fixes this, is say that we now no longer extract 2 bits but need a uniform 1 bit output.

`P(output = 1) = sum of prob of all outcomes with xor 1.`

for two bits, only 01 and 10 give xor 1.

so `P(output = 1) = 2*(0.6*0.4)` and that is 0.48 (much closer to 5 compared to 0.6)

We could have recieved m random bits by dot producting with any random matrix, but generating a `m*n` random seed is costlier than just using a pattern like Toeplitz uses (so its just more efficient way, also notice the pattern of each diagonal having the same element). Another thingy I wanted to talk about the costs of these things, extraction isn't magic it comes at cost.

$$m \leq H_\infty(X) - \underbrace{2\log_2(1/\epsilon)}_{\text{how close to 0 near perfect wish to go}}$$

You can calculate m bits you can extract from here.

min_entropy - some value, for now just abstract it to a value usually 256 (comes out from 2^(-128)) is cryptographically secure. So if I want 256 bits, I'd have to takein around 550 raw bits from qrng. So it adds costs. I have the constraint of giving out exactly n number of bits requested by user, I could never assume what the min entropy of raw bits output from the aer would be, so I just oversample 4 times (i just looked up how to fix the issue and 4 was the ans). There are some extraction methods that use ML techniques, maybe someday I'll add that here asw.